

16^{ème} TFJM - Double et Chiffre - Nombre Permutatif

Erri Vaudry

Mars 2026

1 Introduction

Eléments de résolution pour l'un des problèmes du 16^{ème} *Tournoi Français des Jeunes Mathématiciennes et Mathématiciens (TFJM)*. Le problème en question est le problème numéro 7 *Double et Chiffres* dont on redonne une partie de l'énoncé ci-après.

2 Rappel de l'énoncé

On appelle nombre permutatif un nombre de $\mathbb{N}^*$ dont le double est constitué des mêmes chiffres dans un ordre quelconque, sauf un unique chiffre (compris entre 0 et 4) qui est remplacé par son double.

Par exemple, $2163 = 326$: son double est constitué des mêmes chiffres, sauf un chiffre 1 qui est remplacé par un chiffre 2. En revanche, le nombre 11 n'est pas permutatif, car pour obtenir $211 = 22$ il faudrait remplacer deux de ses chiffres par leurs doubles.

Parmi ceux-là, on appelle :

- nombre rotatif un nombre tel que la permutation consiste en faire passer le premier chiffre en dernier, et c'est ce chiffre qui est doublé. Par exemple, 253 serait rotatif si son double était 534.
- nombre fitator un nombre tel que la permutation consiste en faire passer le dernier chiffre en premier, et c'est ce chiffre qui est doublé. Par exemple, 253 serait fitator si son double était 625.

Le nombre 163 n'est ni rotatif ni fitator.

3 Première partie

3.1 Questions

3.1.1 Question 1

Soit $n \geq 2$

1. Existe-t-il un nombre rotatif à n chiffres
2. Déterminer (ou encadrer aussi précisément que possible) le nombre de nombres rotatifs à n chiffres.

3.1.2 Question 2

Soit $n \geq 2$

1. Existe-t-il un nombre fitator à n chiffres
2. Déterminer (ou encadrer aussi précisément que possible) le nombre de nombres fitators à n chiffres.

3.1.3 Question 3

Soit $n \geq 2$

1. Soit $c \in \{0, 1, 2, \dots, 9\}$. Existe-t-il un nombre permutatif à n chiffres qui finit par le chiffre c ?
2. Déterminer (ou encadrer aussi précisément que possible) le nombre de nombres permutatifs à n chiffres. En particulier, déterminer la limite de ce nombre divisé par $9 \times 10^{n-1}$, si elle existe. ($9 \times 10^{n-1}$ étant le nombre de nombres à n chiffres.)

3.1.4 Question 4

Reprendre la sous-question précédente dans d'autres bases de numérotation.

3.1.5 Question complémentaire

Proposer et étudier d'autres pistes de recherche.

3.2 Proposition de solution

3.2.1 Proposition de solution pour la question 1

Soit le nombre $A = \overline{a_0 \dots a_i \dots a_{n-1}}$ que l'on peut alors écrire $\sum_{i=0}^{n-1} a_i \times 10^{n-1-i}$. Si A est un nombre rotatif alors pour $a_0 \in [1, 4]$ (Cela n'a pas de sens que a_0 le premier chiffre de A valent 0) alors on a le système d'équations et d'inéquations suivant :

$$a_0 \geq 1 \quad (1)$$

$$a_0 \leq 4 \quad (2)$$

$$2 \times A = 2a_0 + \sum_{i=1}^{n-1} a_i \times 10^{n-i} \quad (3)$$

Avec l'équation 3 on cherche à exprimer a_0 en fonction des autres chiffres du nombre A . Après manipulation et simplification de l'équation 3 on obtient 4:

$$a_0 = \frac{4 \sum_{i=1}^{n-1} a_i \times 10^{n-1-i}}{10^{n-1} - 1} \quad (4)$$

Pour $n = 2$ cela donne :

$$a_0 = \frac{4a_1}{9}$$

Sachant que $a_0 \in [1, 4]$, la seule solution dans $\mathbb{N}^*$ possible est que a_1 ait la valeur 9, la valeur de a_0 étant alors 4.

Si on généralise pour $n \geq 2$, on remarque que pour que l'on puisse avoir $a_0 \in [1, 4]$, il faut que chaque a_i avec $i \in [1, n-1]$ ait la valeur 9, ainsi la somme $\sum_{i=1}^{n-1} a_i \times 10^{n-1-i}$ se simplifie avec le dénominateur $10^{n-1} - 1$.

Il existe donc bien au moins un nombre rotatif à n chiffres, et plus précisément pour un $n \geq 2$ donné il existe exactement 1 seul nombre rotatif. Au total, pour $n = 2$, il y a 1 nombre rotatif (49), pour $n = 3$, il y a 2 nombres rotatifs (respectivement à 2 et 3 chiffres, 49 et 499), pour $n \geq 2$, il y a $n - 1$ nombres rotatifs au total.

3.2.2 Proposition de solution pour la question 2

Soit le nombre $A = \overline{a_0 \dots a_i \dots a_{n-1}}$ que l'on peut alors écrire $\sum_{i=0}^{n-1} a_i \times 10^{n-1-i}$. Si A est un nombre rotatif alors pour $a_{n-1} \in [0, 4]$ alors on a le système d'équations et d'inéquations suivant :

$$a_{n-1} \geq 0 \quad (5)$$

$$a_{n-1} \leq 4 \quad (6)$$

$$2 \times A = 2a_{n-1} + \sum_{i=0}^{n-2} a_i \times 10^{n-2-i} \quad (7)$$

Avec l'équation 7 on cherche à exprimer a_{n-1} en fonction des autres chiffres du nombre A . Après manipulation et simplification de l'équation 7 on obtient 8:

$$a_{n-1} = \frac{-19 \sum_{i=0}^{n-2} a_i \times 10^{n-2-i}}{2(1 - 10^{n-1})} \quad (8)$$

Avec $n \geq 2$ et $A \in \mathbb{N}^*$ ce qui implique que tous les a_i ne peuvent pas être nuls en même temps.

On remarque que le dénominateur sera toujours de la forme $2(-9 \dots 9)$ donc un nombre pair, au numérateur on a 19 qui est un nombre premier en facteur. Il faut que a_{n-1} soit entier et que $a_{n-1} \in [0, 4]$. Même en prenant à chaque fois la plus haute valeur possible pour les a_i de $\sum_{i=0}^{n-2} a_i \times 10^{n-2-i}$, la somme totale sera au plus égale (au signe près) à $1 - 10^{n-1}$, donc il restera toujours au moins 2 au dénominateur, et il sera impossible d'avoir une valeur entière dans $[0, 4]$ pour a_{n-1} .

Cette équation ne semble pas avoir de solution entière possible dans $[0, 4]$ pour a_{n-1} .

Il n'existe donc pas de nombres fitators à n chiffres, le nombre de nombres fitator à n chiffres est donc 0.

3.2.3 Proposition de solution pour la question 3

A FAIRE

3.2.4 Proposition de solution pour la question 4

A FAIRE

3.2.5 Proposition pour la question complémentaire

A FAIRE

4 Annexe - Programme Python de recherche de nombres permutatifs

```
def is_rotatif(n: int) -> bool:
    if n < 10:
        return False

    double_n = 2 * n
    str_n = str(n)
    str_double_n = str(double_n)

    if int(str_n[0]) > 4:
        return False

    # An alternative way to do it
    # expected_str_n = str(int(str_double_n[-1]) // 2) + str_double_n[0:-1]
    # return expected_str_n == str_n
    expected_str_double_n = str_n[1:] + str(int(str_n[0]) * 2)
    return expected_str_double_n == str_double_n
```

```

def rotatif_numbers_below(limit: int) -> list[int]:
    return [n for n in range(limit) if is_rotatif(n)]

def is_fitator(n: int) -> bool:
    if n < 10:
        return False

    double_n = 2 * n
    str_n = str(n)
    str_double_n = str(double_n)

    if int(str_n[-1]) > 4:
        return False

    # An alternative way to do it
    # expected_str_n = str_double_n[1:] + str(int(str_double_n[0]) // 2)
    # return expected_str_n == str_n
    expected_str_double_n = str(int(str_n[-1]) * 2) + str_n[0:-1]
    return expected_str_double_n == str_double_n

def fitator_numbers_below(limit: int) -> list[int]:
    return [n for n in range(limit) if is_fitator(n)]

def is_permutatif(n: int) -> bool:
    if n < 10:
        return False

    double_n = 2 * n
    str_n = sorted(str(n))
    str_double_n = sorted(str(double_n))

    expected_str_double_n = []
    count_below_5 = len([digit for digit in str_n if digit in "01234"])
    i = 0
    while i < count_below_5:
        expected_str_double_n.append(
            str_n[0:i] + [str(int(str_n[i]) * 2)] + str_n[i + 1 :]
        )
        i += 1

    return any(

```

```

        expected_str_double_n == str_double_n
        for expected_str_double_n in expected_str_double_n
    )

def permutatif_numbers_below(limit: int) -> list[int]:
    return [n for n in range(limit) if is_permutatif(n)]

def main():
    limit = 100_000
    print("Rotatif-numbers-below", limit, ":", rotatif_numbers_below(limit))
    print("Fitator-numbers-below", limit, ":", fitator_numbers_below(limit))
    print("Permutatif-numbers-below", limit, ":", permutatif_numbers_below(limit))

if __name__ == "__main__":
    main()

```